2HandToYou AI Service

คู่มือการติดตั้งระบบ

Installation Manual

AI Price Service (ประเมินราคาและตรวจเกรดสินค้ามือสอง)
AI ChatBot Service (แชทบอทถาม-ตอบ และแชทบอทเจรจาต่อรอง)

รายการ	รายละเอียด
เวอร์ชันเอกสาร	1.3
เวอร์ชันระบบ	AI Price Service 1.0 · AI ChatBot Service 2.0 · Knowledge Base 2026.10.02
วันที่	ตุลาคม 2569
จัดทำโดย	ทีม AI — โครงการ 2HandToYou
ซอร์สโค้ด	github.com/SupawitMon/ai-service-2hand

ประวัติการแก้ไขเอกสาร

เวอร์ชัน	วันที่	รายละเอียดการแก้ไข	ผู้จัดทำ
1.0	ตุลาคม 2569	จัดทำเอกสารฉบับแรก	ทีม AI
1.1	ตุลาคม 2569	แก้ตามผลรีวิว: หัวข้อ 6.4 Postman ใช้ URL เดียว (base_url พอร์ต 8012) ฝังใน collection และแนบไฟล์ Postman Collection ในชุดส่งมอบ	ทีม AI
1.2	ตุลาคม 2569	แก้ตามผลรีวิว: หัวข้อ 5.3 เพิ่มเกณฑ์และขั้นตอนสร้างรหัส LOG_ACCESS_TOKEN แบบสุ่ม (Linux/Windows) การป้องกันไฟล์ .env การตรวจสอบ และการเปลี่ยนรหัส	ทีม AI
1.3	ตุลาคม 2569	แก้ตามผลรีวิว: base_url ใน Postman collection และ servers ใน API Spec ชี้ไปเซิร์ฟเวอร์จริง http://10.80.35.5:8012 แทน localhost และปรับหัวข้อ 6.4 ให้ตรงกัน	ทีม AI

การตรวจสอบและอนุมัติ

บทบาท	ชื่อ	วันที่
ผู้จัดทำ	ทีม AI	
ผู้ตรวจสอบ		
ผู้อนุมัติ		

สารบัญ

1. บทนำ....4
2. ภาพรวมระบบ....5
3. ความต้องการของระบบ....6
4. รายการไฟล์ที่ส่งมอบ....7
5. ขั้นตอนการติดตั้ง....8
6. การตรวจสอบหลังติดตั้ง....11
7. การตั้งค่าระบบ....13
8. การใช้งานและดูแลระบบ....14
9. การติดตั้งสำหรับนักพัฒนา (ไม่ใช้ Docker)....16
10. การแก้ไขปัญหาเบื้องต้น....17
11. ข้อควรทราบด้านความปลอดภัย....18
ภาคผนวก ก. สรุป API....19
ภาคผนวก ข. คำสั่งที่ใช้บ่อย....20

1. บทนำ

1.1 วัตถุประสงค์

เอกสารนี้อธิบายขั้นตอนการติดตั้ง ตั้งค่า ตรวจสอบ และดูแลระบบ **2HandToYou AI Service** บนเครื่องแม่ข่าย (server) เพื่อให้ผู้ดูแลระบบสามารถติดตั้งระบบให้พร้อมใช้งานได้ด้วยตนเองตั้งแต่ต้นจนจบ

1.2 ขอบเขต

ครอบคลุมบริการ 3 ส่วนที่ทำงานร่วมกันผ่าน Docker Compose ได้แก่ AI Price Service, AI ChatBot Service และ API Gateway (nginx) ซึ่งให้บริการหน้าเว็บสาธิตด้วย รายละเอียดของ API แต่ละตัวอยู่ในเอกสาร API Specification (`openapi.yaml` / `swagger.html`) ที่ส่งมอบคู่กัน

1.3 กลุ่มผู้อ่าน

- ผู้ดูแลระบบ (System Administrator) ที่ติดตั้งและดูแลเครื่องแม่ข่าย
- ทีมพัฒนาระบบ (SE / Developer) ที่นำ API ไปเชื่อมต่อ

ผู้อ่านควรใช้งานคำสั่ง Linux พื้นฐานได้ ไม่จำเป็นต้องมีความรู้ด้าน AI

1.4 คำศัพท์

คำศัพท์	ความหมาย
Docker / Docker Compose	เครื่องมือรันโปรแกรมใน container ทำให้ติดตั้งได้เหมือนกันทุกเครื่อง / ไฟล์ `docker-compose.yml` สั่งรันหลาย container พร้อมกัน
Container	ชุดโปรแกรมที่แยกทำงานอิสระ ระบบนี้มี 3 container
API Gateway (nginx)	ตัวรับคำขอจากภายนอกที่พอร์ต 8012 แล้วส่งต่อไปยังบริการที่ถูกต้อง
Knowledge Base (KB)	ข้อมูลสินค้า ราคา และผู้ขาย ที่แชทบอทใช้ตอบ (โฟลเดอร์ `kb/`)
Intent	เจตนาของคำถาม เช่น ถามราคา ถามสภาพสินค้า
Fallback	การที่แชทบอทไม่ตอบเมื่อไม่มั่นใจหรือไม่มีข้อมูล แล้วถามกลับหรือเสนอทางเลือกแทน
Request ID	รหัสอ้างอิงของแต่ละคำขอ ใช้ค้นหา log ย้อนหลัง

2. ภาพรวมระบบ

2.1 องค์ประกอบของระบบ

Container	บริการ	หน้าที่
`ai-frontend`	API Gateway (nginx)	รับคำขอทั้งหมดที่พอร์ต 8012 ส่งต่อไปยัง 2 บริการด้านล่าง และให้บริการหน้าเว็บสาธิต
`ai-price-backend`	AI Price Service	ทายเกรดสภาพสินค้าจากรูป (YOLO) และประเมินราคา (XGBoost)
`ai-chatbot`	AI ChatBot Service	แชทบอทถาม-ตอบ (จำบริบทตาม session, ตอบจาก KB) และจำแนกข้อความในแชทเจรจาต่อรอง พร้อม request ID, log และ metrics

2.2 แผนภาพการเชื่อมต่อ

```
ผู้ใช้ / ระบบของ SE
      │   http://<IP>:8012
      ▼
ai-frontend  (nginx — API Gateway)
  ├─ /api/options, /api/predict-*   ──►  ai-price-backend (ภายใน :5000)
  ├─ /api/chat, /api/negotiate,     ──►  ai-chatbot (:5001)
  │  /api/metrics, /api/logs, /api/kb
  └─ /  และ /negotiation_chatbot.html  ──►  หน้าเว็บสาธิต

http://<IP>:8013  ──►  ai-chatbot โดยตรง (Swagger ของแชทบอท)
```

2.3 พอร์ตและที่อยู่สำหรับใช้งาน

ที่อยู่	ใช้ทำอะไร
`http://<IP>:8012/`	หน้าเว็บสาธิต AI ประเมินราคา
`http://<IP>:8012/negotiation_chatbot.html`	หน้าเว็บสาธิตแชทบอทเจรจาต่อรอง
`http://<IP>:8012/api/...`	API ทุกตัว (จุดเชื่อมต่อหลักสำหรับระบบอื่น)
`http://<IP>:8012/apidocs/`	Swagger ของ AI Price Service
`http://<IP>:8013/apidocs/`	Swagger ของ AI ChatBot Service

> **หมายเหตุ:** AI Price Service ไม่เปิดพอร์ตออกภายนอกโดยตรง ต้องเรียกผ่านพอร์ต 8012 เท่านั้น

3. ความต้องการของระบบ

3.1 ฮาร์ดแวร์

รายการ	ขั้นต่ำ	แนะนำ
CPU	2 vCPU	4 vCPU
หน่วยความจำ (RAM)	4 GB	8 GB
พื้นที่ดิสก์ว่าง	10 GB	20 GB (เผื่อ log และอัปเดต)
GPU	ไม่จำเป็น (ทำงานด้วย CPU)	ไม่จำเป็น

> **หมายเหตุ:** ค่าข้างต้นเป็นค่าประมาณ Docker image ของ AI Price Service มีขนาดราว 2–3 GB เพราะรวม PyTorch สำหรับโมเดล YOLO

3.2 ซอฟต์แวร์

ซอฟต์แวร์	เวอร์ชัน	หมายเหตุ
ระบบปฏิบัติการ	Ubuntu 22.04 / 24.04 LTS	หรือ Linux อื่นที่รองรับ Docker
Docker Engine	24 ขึ้นไป	ติดตั้งตามหัวข้อ 5.1
Docker Compose	v2 (คำสั่ง `docker compose`)	มาพร้อม Docker Engine รุ่นใหม่
Git	รุ่นใดก็ได้	ใช้ดาวน์โหลดซอร์สโค้ด

> **ข้อควรระวัง:** ถ้าติดตั้งบน Windows ต้องใช้ Docker Desktop และต้องเปิด Virtualization ใน BIOS และฟีเจอร์ WSL 2 ก่อน (ดูหัวข้อ 10) แนะนำให้ใช้เครื่องแม่ข่าย Linux สำหรับใช้งานจริง

3.3 เครือข่าย

- **ขาเข้า:** เปิดพอร์ต **8012** (หน้าเว็บ + API) และ **8013** (แชทบอทโดยตรง) ให้เครื่องที่ต้องใช้งานเข้าถึงได้
- **ขาออก (เฉพาะตอนติดตั้ง/อัปเดต):** ต้องเข้าถึง Docker Hub, PyPI (`pypi.org`), `download.pytorch.org` และ GitHub ได้ เพื่อดาวน์โหลดแพ็กเกจ
- หลังติดตั้งเสร็จ ระบบทำงานได้โดยไม่ต้องต่ออินเทอร์เน็ต

4. รายการไฟล์ที่ส่งมอบ

ซอร์สโค้ดและไฟล์ทั้งหมดอยู่ใน GitHub repository `SupawitMon/ai-service-2hand` โครงสร้างหลักดังนี้

ไฟล์ / โฟลเดอร์	คำอธิบาย
`docker-compose.yml`	ไฟล์สั่งรันทั้ง 3 container
`Dockerfile`, `Dockerfile.chatbot`	วิธีสร้าง image ของ AI Price Service และ AI ChatBot Service
`nginx.conf`	การตั้งค่า API Gateway (เส้นทางของ API)
`.env.example`	ตัวอย่างไฟล์ตั้งค่ารหัสผ่านดู log (คัดลอกเป็น `.env`)
`backend_server.py`	โปรแกรม AI Price Service
`best.pt`	โมเดลทายเกรดจากรูป (YOLO)
`price_model.json`, `price_model_categories.pkl`, `product_lookup.json`	โมเดลประเมินราคา (XGBoost) และข้อมูลราคาตลาดอ้างอิง
`chatbot_server.py`, `chatbot_dataset.json`	โปรแกรม AI ChatBot Service และชุดข้อมูลคำถาม-คำตอบ
`kb/`	Knowledge Base ของแชทบอท (สินค้า ผู้ขาย ตารางราคา ตัวอย่างประโยคสำหรับเทรน)
`mockup_ai_price.html`, `negotiation_chatbot.html`	หน้าเว็บสาธิต
`build_kb.py`, `sync_negotiation_catalog.py`	สคริปต์อัปเดต Knowledge Base
`eval_chatbot.py`, `tests/`, `reports/`	สคริปต์และชุดทดสอบวัดผลแชทบอท พร้อมผลทดสอบล่าสุด
`postman/`	Postman collection สำหรับทดสอบ API (ไฟล์เดียว ใช้ base_url พอร์ต 8012)
`docs/`	เอกสารประกอบ

เอกสารที่ส่งมอบคู่กับคู่มือนี้: **API Specification** (`openapi.yaml`, `openapi.json`, `swagger.html`) และ **Postman Collection** (`2HandToYou_AI_Service.postman_collection.json`)

5. ขั้นตอนการติดตั้ง

ขั้นตอนนี้ใช้กับเครื่องแม่ข่าย Ubuntu ทุกคำสั่งรันใน Terminal ของเครื่องแม่ข่าย

5.1 ติดตั้ง Docker

ข้ามขั้นตอนนี้ได้ถ้าเครื่องมี Docker อยู่แล้ว (ตรวจด้วย `docker --version`)

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

จากนั้น **ออกจากระบบแล้วเข้าใหม่ 1 ครั้ง** เพื่อให้ใช้คำสั่ง docker ได้โดยไม่ต้องพิมพ์ sudo แล้วตรวจสอบว่าทั้ง 2 คำสั่งแสดงเลขเวอร์ชัน:

```
docker --version
docker compose version
```

5.2 ดาวน์โหลดซอร์สโค้ด

```
cd ~
git clone https://github.com/SupawitMon/ai-service-2hand.git
cd ai-service-2hand
```

> **หมายเหตุ:** ถ้าได้รับไฟล์เป็น .zip แทน ให้แตกไฟล์ไว้ในโฟลเดอร์เดียว แล้ว cd เข้าไปในโฟลเดอร์นั้น ต้องเห็นไฟล์ docker-compose.yml

5.3 ตั้งค่าไฟล์ .env และสร้างรหัส LOG_ACCESS_TOKEN

ไฟล์ `.env` เก็บรหัส `LOG_ACCESS_TOKEN` ซึ่งเป็นรหัสผู้ดูแลระบบสำหรับเปิดดู log ย้อนหลังผ่าน API `GET /api/logs/<request_id>` (log มีข้อความที่ผู้ใช้พิมพ์ จึงต้องป้องกันไม่ให้คนอื่นเปิดได้) ขั้นตอนนี้ไม่บังคับ ถ้าไม่ตั้ง ระบบยังทำงานปกติ และ endpoint ดู log จะถูกปิดอัตโนมัติ (ตอบ 403 ทุกครั้ง)

เกณฑ์ของรหัสที่ใช้ได้

- สร้างจากตัวสุ่มของระบบ (ไม่ตั้งเอง ไม่ใช้ชื่อ วันเกิด หรือคำในพจนานุกรม)
- ยาวอย่างน้อย 32 ตัวอักษร (แนะนำ 64 ตัวอักษร hex = 256 บิต) ใช้เฉพาะ 0-9 a-z A-Z เพื่อไม่ให้มีอักขระพิเศษที่ทำให้ไฟล์ .env อ่านผิด
- ห้ามใช้ค่าตัวอย่าง `change-me` และห้ามใช้รหัสเดียวกับระบบอื่น ใช้แยกกันแต่ละเครื่อง (เครื่องทดสอบกับเครื่องจริงคนละรหัส)

ขั้นที่ 1: คัดลอกไฟล์ตัวอย่าง

```
cp .env.example .env
```

ขั้นที่ 2: สร้างรหัสสุ่ม (เลือกวิธีใดวิธีหนึ่ง)

Linux / macOS (มี openssl ติดมากับระบบ):

```
openssl rand -hex 32
```

ถ้าไม่มี openssl ใช้ Python แทน:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Windows (PowerShell ใช้ตัวสุ่มสำหรับงานความปลอดภัย):

```
$b = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
($b | ForEach-Object { $_.ToString("x2") }) -join ""
```

ผลที่ได้เป็นตัวอักษร 0-9 a-f ยาว 64 ตัว ทุกครั้งที่รันจะได้ค่าใหม่ ห้ามใช้ค่าที่เห็นในตัวอย่างหรือในเอกสารนี้

ขั้นที่ 3: ใส่รหัสลงไฟล์ .env

บน Linux ใช้คำสั่งเดียวสร้างรหัสและเขียนลงไฟล์ได้เลย (รหัสจะไม่แสดงบนจอและไม่ค้างใน history):

```
sed -i "s/^LOG_ACCESS_TOKEN=.*/LOG_ACCESS_TOKEN=$(openssl rand -hex 32)/" .env
```

หรือเปิดแก้ด้วย `nano .env` แล้วแทน `change-me` ด้วยรหัสจากขั้นที่ 2 ให้ได้รูปแบบ `LOG_ACCESS_TOKEN=<รหัส 64 ตัว>` (ไม่มีช่องว่าง ไม่ใส่เครื่องหมายคำพูด) แล้วบันทึก Ctrl+O, Enter, Ctrl+X

ขั้นที่ 4: ป้องกันไฟล์ .env

```
chmod 600 .env
grep -c "change-me" .env    # ต้องได้ 0
```

คำสั่งแรกทำให้เฉพาะเจ้าของเครื่องอ่านไฟล์ได้ คำสั่งที่สองตรวจว่าไม่เหลือค่าตัวอย่าง ไฟล์ `.env` อยู่ใน .gitignore แล้ว ห้าม commit ขึ้น GitHub และห้ามส่งรหัสผ่านแชทกลุ่ม ให้ส่งเฉพาะผู้ดูแลระบบที่ต้องใช้

ขั้นที่ 5: ตรวจว่ารหัสใช้งานได้ (ทำหลังขั้นที่ 5.4 เริ่มระบบแล้ว)

```
TOKEN=$(grep ^LOG_ACCESS_TOKEN= .env | cut -d= -f2)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8012/api/logs/test
curl -s -o /dev/null -w "%{http_code}\n" -H "X-Admin-Token: $TOKEN"
http://localhost:8012/api/logs/test
```

ผลที่ถูกต้อง: คำสั่งแรก (ไม่ส่งรหัส) ได้ `403` คำสั่งที่สอง (ส่งรหัสถูก) ได้ `404` ซึ่งแปลว่าผ่านการตรวจรหัสแล้ว แต่ไม่พบ request_id ชื่อ test (ปกติ) ถ้าคำสั่งที่สองยังได้ 403 แปลว่า container ยังไม่ได้อ่านค่าใหม่ ให้ทำขั้นถัดไป

การเปลี่ยนรหัส

ควรเปลี่ยนรหัสเมื่อมีคนออกจากทีม เมื่อสงสัยว่ารหัสหลุด หรือทุก 90 วัน ทำขั้นที่ 3 ใหม่ แล้วสั่งให้แชทบอทอ่านค่าใหม่ (ต้องใช้ up -d ไม่ใช่ restart เพราะ restart จะไม่อ่านไฟล์ .env ใหม่):

```
docker compose up -d ai-chatbot
```

5.4 สร้างและเริ่มระบบ

```
docker compose up -d --build
```

ครั้งแรกใช้เวลาประมาณ **5–15 นาที** (ขึ้นกับความเร็วอินเทอร์เน็ต) เพราะต้องดาวน์โหลดและติดตั้งแพ็กเกจ ครั้งต่อไปจะเร็วขึ้นมาก

5.5 เปิดพอร์ตใน Firewall

ถ้าเครื่องเปิด firewall (ufw) อยู่ ให้อนุญาตพอร์ต 8012 และ 8013 (ถ้าเครื่องอยู่หลัง firewall หรือใช้ port mapping ของหน่วยงาน ต้องแจ้งผู้ดูแลเครือข่ายให้เปิด/map 2 พอร์ตนี้มาที่เครื่องนี้ด้วย):

```
sudo ufw allow 8012/tcp
sudo ufw allow 8013/tcp
```

6. การตรวจสอบหลังติดตั้ง

6.1 ตรวจสถานะ container

```
docker compose ps
```

ต้องเห็น 3 container สถานะ **Up** โดย `ai-price-backend` และ `ai-chatbot` ต้องขึ้น **(healthy)** (รอประมาณ 1 นาทีหลังเริ่มระบบ)

6.2 รายการตรวจสอบ

เปิดจากเครื่องอื่นในเครือข่าย โดยแทน `<IP>` ด้วย IP ของเครื่องแม่ข่าย

ลำดับ	ตรวจสอบ	ผลที่ถูกต้อง
1	เปิด `http://<IP>:8012/`	หน้า "AI ประเมินราคาสินค้ามือสอง" และช่องยี่ห้อมีรายการให้เลือก
2	เลือก Apple → iPhone 13 → เกรด B แล้วกดประเมินราคา	ได้ช่วงราคาและกราฟปัจจัยที่มีผลต่อราคา
3	อัปโหลดรูปสินค้าในหน้าเดียวกัน	ระบบแสดงเกรด A/B/C และความมั่นใจ
4	เปิด `http://<IP>:8012/negotiation_chatbot.html` แล้วพิมพ์ "เครื่องมีรอยไหม"	บอทตอบข้อมูลสินค้า และมีบรรทัดเล็ก `intent: ... · request_id ...` ใต้ข้อความ
5	เปิด `http://<IP>:8012/apidocs/`	Swagger ของ AI Price Service
6	เปิด `http://<IP>:8013/apidocs/`	Swagger ของ AI ChatBot Service
7	เปิด `http://<IP>:8012/api/kb/info`	ข้อมูล KB เช่น `"listings": 430`

6.3 ทดสอบ API ด้วยคำสั่ง curl

```
curl http://<IP>:8012/api/options

curl -X POST http://<IP>:8012/api/predict-price \
  -H 'Content-Type: application/json' \
  -d '{"brand":"Apple","model":"iPhone 13","grade":"B"}'

curl -X POST http://<IP>:8012/api/chat \
  -H 'Content-Type: application/json' \
  -d '{"message":"iPhone 13 ราคาเท่าไหร่","session_id":"test1"}'
```

คำสั่งแรกต้องได้รายชื่อยี่ห้อ คำสั่งที่ 2 ต้องได้ `recommended_min` / `recommended_max` และคำสั่งที่ 3 ต้องได้ `response` และ `request_id`

6.4 ทดสอบด้วย Postman

ไฟล์ Postman collection อยู่ในชุดเอกสารที่ส่งมอบ โฟลเดอร์ **3_Postman_Collection** (และในโฟลเดอร์ `postman/` ของซอร์สโค้ด) เป็น**ไฟล์เดียว** คือ `2HandToYou_AI_Service.postman_collection.json` ไม่ต้องใช้ไฟล์ environment

ทุก request (22 รายการ) เรียกผ่าน **API Gateway พอร์ต 8012 เส้นเดียว** ผ่านตัวแปร `base_url` ที่ฝังไว้ใน collection แล้ว ค่าเริ่มต้นคือที่อยู่เซิร์ฟเวอร์จริงของโครงการ `http://10.80.35.5:8012`

1. เปิด Postman → **Import** → เลือกไฟล์ `2HandToYou_AI_Service.postman_collection.json`
2. ทดสอบกับเซิร์ฟเวอร์โครงการ ใช้ค่าเริ่มต้นได้เลยไม่ต้องแก้ ถ้าติดตั้งระบบเองบนเครื่องอื่นตามคู่มือนี้: คลิกชื่อ collection → แท็บ **Variables** → แก้ช่อง **Value** ของ `base_url` เป็น `http://<IP เครื่องนั้น>:8012 (หรือ http://localhost:8012 ถ้ารัน Postman บนเครื่องนั้นเอง)` → **Save**
3. คลิกขวาที่ collection → **Run collection** → **Run** ผลที่ถูกต้อง: 22 request ผ่านทุก test (ข้อ "ทายเกรดจากรูป" จะถูกข้ามถ้ายังไม่ได้เลือกไฟล์รูปใน Body → form-data → `image`)

> **หมายเหตุ:** ระบบใช้ URL เดียวคือ `http://<IP>:8012` สำหรับ API ทุกตัว (nginx ส่งต่อให้แต่ละบริการเอง) พอร์ต 5000/5001 เป็นพอร์ตภายใน container เรียกจากภายนอกไม่ได้ ส่วนพอร์ต 8013 ใช้เปิด Swagger ของแชทบอทเท่านั้น

6.5 วัดผลแชทบอทกับเซิร์ฟเวอร์จริง (ทางเลือก)

รันจากเครื่องที่มี Python และซอร์สโค้ดชุดเดียวกัน เพื่อวัดความถูกต้องและ latency ที่รวมเวลาเครือข่าย:

```
pip install requests
python eval_chatbot.py --url http://<IP>:8012
```

ผลจะอยู่ที่ `reports/eval_report.md` ทุกหัวข้อในตารางสรุปต้องขึ้น "ผ่าน"

7. การตั้งค่าระบบ

7.1 ตัวแปรสภาพแวดล้อม (Environment Variables)

ตั้งค่าได้ในไฟล์ `docker-compose.yml` (หัวข้อ `environment` ของ service `chatbot`) หรือในไฟล์ `.env` หลังแก้ไขต้องรัน `docker compose up -d` อีกครั้ง

ตัวแปร	ค่าเริ่มต้น	ความหมาย
`LOG_ACCESS_TOKEN`	(ว่าง)	รหัสสำหรับเรียก `/api/logs/<request_id>` ถ้าว่าง endpoint นี้จะปิด
`FALLBACK_THRESHOLD`	0.25	ความมั่นใจต่ำกว่าค่านี้ แชทบอทจะไม่ตอบและถามกลับแทน (ค่ามากขึ้น = ระวังมากขึ้น แต่ตอบน้อยลง)
`SESSION_TTL_SEC`	1800	ระยะเวลาที่จำบริบทการสนทนา (วินาที)
`LOG_RETENTION_DAYS`	30	จำนวนวันที่เก็บ log ย้อนหลัง
`HIGH_CONFIDENCE`	0.60	เกณฑ์ความมั่นใจระดับ high ใน `confidence_level`
`FLASK_DEBUG`	0	ต้องเป็น 0 เสมอบนเซิร์ฟเวอร์จริง

7.2 เปลี่ยนพอร์ต

แก้ในไฟล์ `docker-compose.yml` เฉพาะตัวเลข **ด้านซ้าย** ของเครื่องหมาย `:` เช่น เปลี่ยน `"8012:80"` เป็น `"9000:80"` แล้วรัน `docker compose up -d`

7.3 เส้นทางของ API Gateway

กำหนดในไฟล์ `nginx.conf`: เส้นทาง `/api/chat`, `/api/negotiate`, `/api/metrics`, `/api/logs`, `/api/kb` ส่งไปที่ AI ChatBot Service ส่วน `/api/` ที่เหลือและ `/apidocs` ส่งไปที่ AI Price Service ขนาดไฟล์รูปที่อัปโหลดได้สูงสุด 20 MB (`client_max_body_size`)

8. การใช้งานและดูแลระบบ

8.1 คำสั่งควบคุมระบบ

รันในโฟลเดอร์ `ai-service-2hand`

งาน	คำสั่ง
เริ่มระบบ	`docker compose up -d`
หยุดระบบ	`docker compose down`
รีสตาร์ททั้งหมด	`docker compose restart`
รีสตาร์ทเฉพาะแชทบอท	`docker compose restart chatbot`
ดูสถานะ	`docker compose ps`
ดู log แบบต่อเนื่อง	`docker compose logs -f backend` (เปลี่ยนเป็น `chatbot` หรือ `frontend` ได้)

ทุก container ตั้งค่า `restart: unless-stopped` ระบบจะเริ่มทำงานเองหลังเครื่องรีบูต

8.2 อัปเดตระบบเป็นเวอร์ชันใหม่

```
cd ~/ai-service-2hand
git pull
docker compose up -d --build
```

ไฟล์ `.env` และโฟลเดอร์ `logs/` จะไม่ถูกเขียนทับ

8.3 อัปเดตข้อมูลสินค้าและราคาของแชทบอท

1. แก้ราคามือสองต่อรุ่นในไฟล์ `kb/price_table.json` หรือข้อมูลสินค้า/ผู้ขายใน `chatbot_dataset.json`
2. รัน `python3 build_kb.py` และ `python3 sync_negotiation_catalog.py` (ต้องมี Python 3)
3. ตรวจผลด้วย `python3 eval_chatbot.py` (ต้องติดตั้งแพ็กเกจใน `requirements_chatbot.txt` ก่อน)
4. รัน `docker compose up -d --build chatbot frontend`

8.4 เปลี่ยนโมเดลทายเกรด

นำไฟล์โมเดลใหม่ (ไฟล์ `.pt` จากการเทรนด้วย `train_grade_v4.py`) มาวางทับ `best.pt` แล้วรัน `docker compose up -d --build backend`

8.5 Log ของแชทบอท

- ทุกคำขอถูกบันทึกในไฟล์ `logs/chat_requests.jsonl` บรรทัดละ 1 คำขอ (เวลา, request_id, ข้อความ, intent, ความมั่นใจ, แหล่งข้อมูลที่ใช้ตอบ, คำตอบ, เวลาประมวลผล)
- เบอร์โทรศัพท์และอีเมลในข้อความถูกแทนด้วย `[PHONE]` / `[EMAIL]` ก่อนบันทึก
- หมุนไฟล์ทุกเที่ยงคืน และลบไฟล์ที่เก่ากว่า `LOG_RETENTION_DAYS` วันโดยอัตโนมัติ

- ค้นหา log ด้วย request_id:

```
curl -H 'X-Admin-Token: <รหัสใน .env>' \
  http://<IP>:8012/api/logs/<request_id>
```

8.6 ตรวจประสิทธิภาพ

เปิด `http://<IP>:8012/api/metrics` เพื่อดูจำนวนคำขอ, latency (P50/P95/P99) และอัตรา fallback ตั้งแต่เริ่มระบบ (ค่าจะรีเซ็ตเมื่อรีสตาร์ท)

8.7 การสำรองข้อมูล

ควรสำรองไฟล์ต่อไปนี้เป็นประจำ: ไฟล์ `.env`, โฟลเดอร์ `logs/` และไฟล์ที่แก้ไขเอง (เช่น `kb/price_table.json`, `best.pt`) ส่วนซอร์สโค้ดอื่นดาวน์โหลดใหม่จาก GitHub ได้

9. การติดตั้งสำหรับนักพัฒนา (ไม่ใช้ Docker)

ใช้สำหรับพัฒนาหรือทดสอบบนเครื่องส่วนตัว (Windows / macOS / Linux) โดยไม่ใช้ Docker ต้องมี **Python 3.10 ขึ้นไป** (ทดสอบแล้วกับ 3.11, 3.12 และ 3.14)

9.1 ติดตั้ง

```
python -m venv .venv
# Windows:  .venv\Scripts\Activate.ps1
# Linux/macOS:  source .venv/bin/activate
pip install -r requirements_backend.txt -r requirements_chatbot.txt
```

9.2 รันบริการ

เปิด Terminal 2 หน้าต่าง (เปิด virtual environment ทั้งคู่) แล้วรันหน้าต่างละ 1 คำสั่ง:

```
python backend_server.py      # AI Price Service  → http://localhost:5000
python chatbot_server.py      # AI ChatBot Service → http://localhost:5001
```

เปิดไฟล์ `mockup_ai_price.html` หรือ `negotiation_chatbot.html` ด้วยการดับเบิลคลิก หน้าเว็บจะเรียก API ที่ localhost ให้เอง

> **หมายเหตุ:** ในโหมดนี้หน้าแชทเจรจาจะใช้ตรรกะสำรองในหน้าเว็บ (ไม่เรียก /api/negotiate/intent) เพราะไม่มี API Gateway การทำงานเต็มรูปแบบต้องติดตั้งด้วย Docker ตามหัวข้อ 5

10. การแก้ไขปัญหาเบื้องต้น

อาการ	สาเหตุ	วิธีแก้
`docker: command not found` หรือ `docker is not recognized`	ยังไม่ได้ติดตั้ง Docker	ติดตั้งตามหัวข้อ 5.1 (Windows: ติดตั้ง Docker Desktop แล้วเปิด Terminal ใหม่)
`permission denied ... docker.sock`	ผู้ใช้ยังไม่อยู่ในกลุ่ม docker	รัน `sudo usermod -aG docker $USER` แล้วออกจากระบบแล้วเข้าใหม่
Windows: Docker Desktop ขึ้น "Virtualization support not detected" หรือ Engine เปิดแล้วดับ	ไม่ได้เปิด Virtualization หรือฟีเจอร์ WSL 2 / Virtual Machine Platform ติดตั้งไม่สมบูรณ์	เปิด Intel VT-x / AMD SVM ใน BIOS, รัน `wsl --update` และเปิดฟีเจอร์ Virtual Machine Platform แล้วรีสตาร์ท ถ้ายังไม่ได้แนะนำให้ติดตั้งบนเครื่องแม่ข่าย Linux แทน
`port is already allocated`	มีโปรแกรมอื่นใช้พอร์ต 8012 หรือ 8013	ปิดโปรแกรมนั้น หรือเปลี่ยนพอร์ตตามหัวข้อ 7.2
Build ล้มเหลวระหว่างดาวน์โหลดแพ็กเกจ	เครื่องเข้าอินเทอร์เน็ตไม่ได้ หรือถูก proxy/firewall บล็อก	ตรวจการเข้าถึงตามหัวข้อ 3.3 แล้วรัน `docker compose up -d --build` ใหม่
เปิดหน้าเว็บจากเครื่องอื่นไม่ได้ แต่ในเครื่องแม่ข่ายเปิดได้	Firewall หรือ port mapping ยังไม่เปิด	ทำตามหัวข้อ 5.5 หรือแจ้งผู้ดูแลเครือข่าย
ช่องยี่ห้อขึ้น "โหลดรายชื่อไม่ได้"	AI Price Service ยังไม่พร้อม	รอ 1–2 นาทีหลังเริ่มระบบ แล้วดู `docker compose logs backend`
ทายเกรดจากรูปตอบ error 500 "YOLO model ยังไม่ได้โหลด"	ไฟล์ `best.pt` หายหรือเสีย	ตรวจว่ามีไฟล์ `best.pt` ในโฟลเดอร์ แล้วรัน `docker compose up -d --build backend` (การประเมินราคายังใช้ได้ปกติ)
ประเมินราคาตอบ 404 "ไม่พบรุ่น ..."	รุ่นนั้นไม่มีในข้อมูลอ้างอิง	เลือกรุ่นจาก `suggestions` ในคำตอบ หรือเพิ่มรุ่นใน `product_lookup.json`
log ของ AI Price Service มีข้อความ "ข้าม category เพี้ยน (invalid UTF-8)"	ข้อมูลบางค่าในไฟล์โมเดลราคามีปัญหาการเข้ารหัส	เป็นคำเตือน ระบบยังทำงานได้ แก้ถาวรได้ด้วยการเทรนโมเดลราคาใหม่
`/api/logs/...` ตอบ 403	ไม่ได้ตั้ง `LOG_ACCESS_TOKEN` หรือส่ง `X-Admin-Token` ไม่ตรง	ตั้งค่าตามหัวข้อ 5.3 แล้วรัน `docker compose up -d`
แชทบอทตอบ "ยังไม่แน่ใจ" บ่อยเกินไป	ค่า `FALLBACK_THRESHOLD` สูงเกินไป	ลดค่าลง (เช่น 0.20) ตามหัวข้อ 7.1 แล้ววัดผลซ้ำด้วย `eval_chatbot.py`

11. ข้อควรทราบด้านความปลอดภัย

- ตั้ง `FLASK_DEBUG=0` เสมอบนเซิร์ฟเวอร์จริง (ค่าเริ่มต้นใน `docker-compose.yml` ตั้งไว้แล้ว)
- ห้าม commit ไฟล์ `.env` และโฟลเดอร์ `logs/` ขึ้น Git (ไฟล์ `.gitignore` กันไว้แล้ว)
- ตั้ง `LOG_ACCESS_TOKEN` ให้คาดเดายาก และให้เฉพาะผู้ดูแลระบบเท่านั้น
- API ยังไม่มีระบบยืนยันตัวตน ควรเปิดพอร์ต 8012/8013 เฉพาะในเครือข่ายที่เชื่อถือได้ หรือวางหลังระบบยืนยันตัวตน/Reverse Proxy ของหน่วยงาน
- แชทบอทปฏิเสธคำขอข้อมูลติดต่อส่วนตัวของผู้ขายเสมอ (นโยบาย BR003) และปิดบังเบอร์โทร/อีเมลใน log
- ลิงก์ชั่วคราวประเภท Cloudflare Quick Tunnel (`*.trycloudflare.com`) เหมาะกับการสาธิตเท่านั้น เพราะลิงก์เปลี่ยนทุกครั้งที่เริ่มใหม่

ภาคผนวก ก. สรุป API

รายละเอียด request / response ครบทุกฟิลด์อยู่ในเอกสาร API Specification (`openapi.yaml` หรือเปิด `swagger.html` ด้วยเบราว์เซอร์)

Method	Path	บริการ	ใช้ทำอะไร
GET	`/api/options`	Price	รายชื่อยี่ห้อ/รุ่นทั้งหมด
POST	`/api/predict-grade`	Price	ทายเกรด A/B/C จากรูป (form-data `image`)
POST	`/api/predict-price`	Price	ประเมินช่วงราคา + ปัจจัยที่มีผลต่อราคา
POST	`/api/chat`	ChatBot	ถามแชทบอท (รองรับ `session_id`)
POST	`/api/negotiate/intent`	ChatBot	จำแนกข้อความในแชทเจรจา
GET	`/api/metrics`	ChatBot	latency, จำนวนคำขอ, อัตรา fallback
GET	`/api/kb/info`	ChatBot	เวอร์ชัน KB และค่าตั้ง
GET	`/api/logs/{request_id}`	ChatBot	ดู log (ต้องใช้ `X-Admin-Token`)
GET	`/api/chat/stats`	ChatBot	สถิติข้อมูลแชทบอท
GET	`/api/chat/sample-questions`	ChatBot	ตัวอย่างคำถาม

ภาคผนวก ข. คำสั่งที่ใช้บ่อย

```
# ติดตั้งครั้งแรก
git clone https://github.com/SupawitMon/ai-service-2hand.git
cd ai-service-2hand
cp .env.example .env            # แล้วแก้ LOG_ACCESS_TOKEN
docker compose up -d --build

# ดูสถานะ / log
docker compose ps
docker compose logs -f chatbot

# หยุด / เริ่ม / รีสตาร์ท
docker compose down
docker compose up -d
docker compose restart

# อัปเดตเวอร์ชันใหม่
git pull && docker compose up -d --build

# ตรวจว่าระบบทำงาน
curl http://localhost:8012/api/kb/info
```